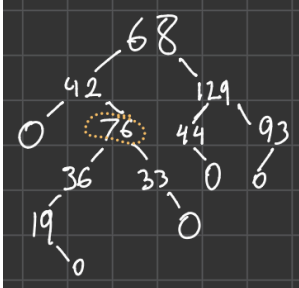
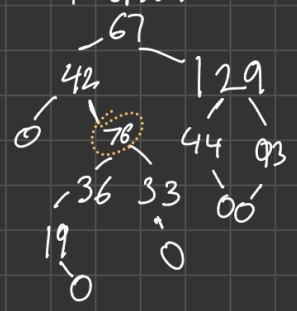
2) Pre order



Inorder

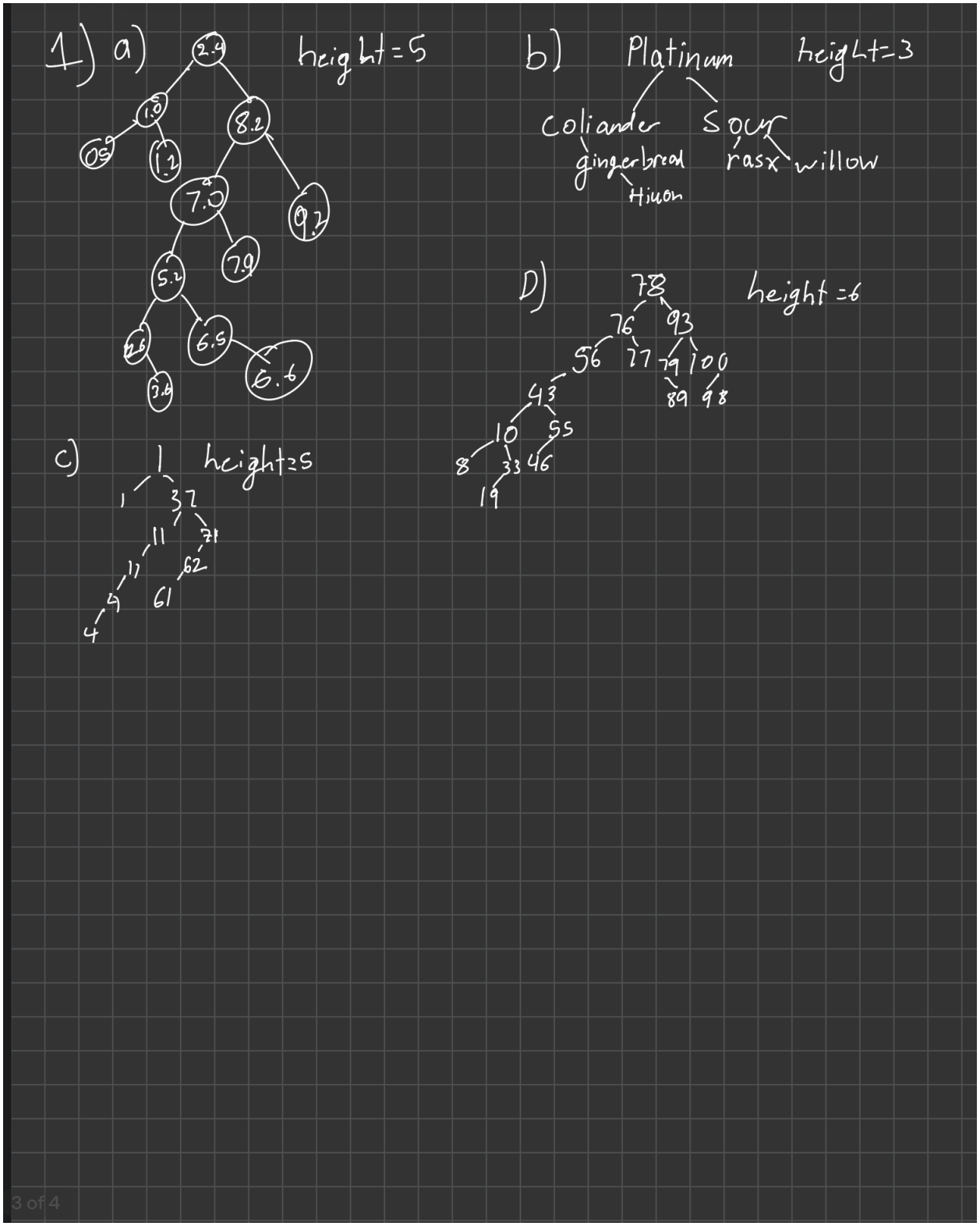


b. are These BST Trees?

No binary search trees needs all nodes in the left subtree to be less than the root. 76 in both trees break it.

C. Are these trees AVL?

NO, The trees must be a valid BST



7) Algorithm Analysis:

3) We analyze this based on the number of unique characters (n) and the total length of the input text (L).

- frequencyCount(String text): O(L) We iterate through the entire string once to populate the frequency map.
- buildHuffman(): O(n^2).
    - We iterate n-1 times to merge nodes until one root remains.
    - In each iteration, we call findMinIndex twice. Since we are using nodeList.remove(), each search and removal in an ArrayList takes O(n) time where n is the length of the array.
    - Total: (n-1) * O(n) = O(n^2).
- Overall Time Complexity: O(L + n^2).
    - Big-O (Worst Case): O(L + n^2). In the worst case, n (unique characters) can be up to 256 (for standard ASCII).

  The space complexity is O(L + n). We must store the encoded output and the tree structure.

4)

- The time complexity is O(N), where N is the length of the array input array, bt. We visit each index in the array exactly once.
- Space Complexity: Big-O: O(N). We create a second array unencrypted of the same size as the input to store calculated values.

5)

Time Complexity is O(N) where N is the length of the input array and we must traverse the entire array once because the Max Heap is ordered by alphabet, not length.

Space complexity is also O(N) where N is the length of the array because at most we need to store every string into our output array.

6)

- Big-O O (K log N): Collecting the K nodes in the range takes O(N) time wh. Each of the K deletions is guaranteed to be O(log n) because the Red-Black rebalancing (rotations and recoloring) ensures the tree height remains logarithmic.
- Space Complexity O(N) : The array stores the K values to be deleted, and the recursion stack for traversal depends on the tree height.